

Application Note: Current Exact Code Path for Tag Read/Filter/Write

Date: 2026-05-21

This document is rewritten to match the exact current implementation in:

- app/src/main/java/com/zebra/rfid/demo/sdksample/RFIDHandler.kt
- app/src/main/java/com/zebra/rfid/demo/sdksample/MainActivity.kt

RFIDHandler Exact Methods

applyTagFilter

```
```kotlin
public fun applyTagFilter(tag: String) {
 val currentReader = reader
 if (currentReader == null) {
 Log.e(TAG, "RFIDReader is null")
 return
 }
 //E280119120007676B2EF034A000000009000

 // Add state aware pre-filter
 val filters = PreFilters()
 // In Kotlin, inner classes are instantiated by calling the constructor on the
 // outer instance
 val filter = filters.PreFilter()

 filter.setAntennaID(1.toShort()) // Explicitly cast to Short in Kotlin
 filter.setTagPattern(tag)
 if(tag.length*4 > 128)
 filter.setTagPatternBitCount(128)
 else
 filter.setTagPatternBitCount(tag.length * 4)

 filter.setBitOffset(0) // skip PC bits (always it should be in bit length)
 filter.setMemoryBank(MEMORY_BANK.MEMORY_BANK_TID)
 filter.setFilterAction(FILTER_ACTION.FILTER_ACTION_STATE_AWARE)

 // inventoried flag of session S1 of matching tags to B
 filter.StateAwareAction.setTarget(TARGET.TARGET_INVENTORIED_STATE_S0)

 // not to select tags that match the criteria
 filter.StateAwareAction.setStateAwareAction(STATE_AWARE_ACTION.STATE_AWARE_A
 CTION_INV_B_NOT_INV_A)

 try {
 // Clear existing pre-filters before adding new ones
 currentReader.Actions.PreFilters.deleteAll()

 // Add the new filter
 currentReader.Actions.PreFilters.add(filter)
 Log.i(TAG, "State-aware pre-filter added successfully for tag pattern: $
tag")
 } catch (e: InvalidUsageException) {
 Log.e(TAG, "InvalidUsageException: ${e.vendorMessage}")
 e.printStackTrace()
 } catch (e: OperationFailureException) {
 Log.e(TAG, "OperationFailureException: ${e.vendorMessage} Status: ${e.re
sults}")
 e.printStackTrace()
 }
}
```

```
}...
```

```
setSingulationForFilter
```

```
```kotlin
public fun setSingulationForFilter(bSet: Boolean) {
    try {
        // Use safe call ?. to ensure reader is not null
        val s0SingulationControl = reader?.Config?.Antennas?.getSingulationContr
ol(1) ?: return

        // Kotlin property access for setters
        s0SingulationControl.session = SESSION.SESSION_S0

        // Accessing the Action nested object
        if(bSet)
            s0SingulationControl.Action.inventoryState = INVENTORY_STATE.INVENTO
RY_STATE_B
        else
            s0SingulationControl.Action.inventoryState = INVENTORY_STATE.INVENTO
RY_STATE_A

        s0SingulationControl.Action.setPerformStateAwareSingulationAction(true)
        // Apply the configuration back to the reader
        reader?.Config?.Antennas?.setSingulationControl(1, s0SingulationControl)

    } catch (e: InvalidUsageException) {
        e.printStackTrace()
    } catch (e: OperationFailureException) {
        e.printStackTrace()
    }
}
}...
```

```
### write
```

```
```kotlin
public fun write(selectTagID: String, dataToWrite: String, bank: MEMORY_BANK, wo
rdOffset: Int) {
 if (executor.isShutdown) {
 executor = Executors.newSingleThreadExecutor()
 }
 executor.execute {
 if (dataToWrite.isEmpty() || dataToWrite.length % 4 != 0) {
 context?.sendToast("Write Failed: Invalid data length")
 return@execute
 }
 try {
 if (isReaderConnected()) {
 reader?.Actions?.Inventory?.stop()
 Thread.sleep(100)
 }
 } catch (e: Exception) {
 Log.e(TAG, "Stop inventory failed", e)
 }

 val tagData = TagData()
 val tagAccess = TagAccess()
 val writeAccessParams = tagAccess.WriteAccessParams()

 writeAccessParams.apply {
```



```
Log.d(TAG, "Start readWait for tagEPC=$selectTagID, bank=${readAccessParams.memoryBank}, offset=$wordOffset, wordCount=${readAccessParams.count}")
// Set useInventoryFilter to true to respect Pre-Filters and Singulation settings (e.g., Session S0, State B)
val effectiveTagID = if (selectTagID.isEmpty()) null else selectTagID

D val tagData = reader?.Actions?.TagAccess?.readWait(effectiveTagID, readAccessParams, null, true)
 val result = tagData?.memoryBankData
 context?.runOnUiThread { callback(result) }
} catch (e: OperationFailureException) {
 Log.e(TAG, "Read failed: ${e.vendorMessage} ${e.results}", e)
 context?.sendToast("Read Failed: ${e.results}")
 context?.runOnUiThread { callback(null) }
} catch (e: Exception) {
 Log.e(TAG, "Read failed", e)
 context?.sendToast("Read Failed: ${e.message}")
 context?.runOnUiThread { callback(null) }
}

}
```